

# security-scan — Guia Completo de Uso

Versão 0.1.3 | Node.js  $\geq 24$  | Docker obrigatório

## Começando em 30 segundos

**Docker obrigatório** — nenhuma outra dependência de runtime precisa ser instalada localmente.

```
# 1. Instalar
npm install -g security-scan

# 2. Gerar configuração (assistente interativo)
security-scan init

# 3. Varrer vulnerabilidades (somente leitura)
security-scan scan

# 4. Aplicar correções seguras automaticamente
security-scan fix
```

## Sumário

1. [Visão Geral](#)
2. [Requisitos](#)
3. [Instalação](#)
4. [Início Rápido](#)
5. [Comandos](#)
  - [init](#)
  - [scan](#)
  - [fix](#)
  - [executive-report](#)
  - [cloud-setup](#)
6. [Referência de Configuração](#)
  - [project](#)
  - [report\\_language](#)
  - [ecosystems](#)
  - [protected\\_packages](#)
  - [safe\\_update\\_policy](#)
  - [conflict\\_resolution](#)

- [scanners](#)
  - [runners](#)
  - [scan](#) (caminhos de varredura)
  - [outputs](#)
  - [cloud\\_storage](#)
  - [workflow](#)
7. [Docker e Estratégias de Runtime](#)
    - [Image Source: pull vs dockerfile](#)
    - [Resolução de Versão do Runtime](#)
    - [Dependências Nativas de SO](#)
  8. [Engines de Scanner](#)
    - [OSV Scanner](#)
    - [SonarQube](#)
  9. [Plugins de Ecossistema e Estratégias de Fix](#)
    - [npm](#)
    - [composer](#)
    - [pip](#)
    - [Estratégias de Fix](#)
  10. [Pacotes Protegidos e Política de Atualização Segura](#)
  11. [Fluxo de Branch e PR no Git](#)
  12. [Integração com CI/CD](#)
  13. [Variáveis de Ambiente](#)
  14. [Códigos de Saída](#)
  15. [Solução de Problemas](#)
  16. [Perguntas Frequentes](#)

## Visão Geral

`security-scan` é uma ferramenta de linha de comando que automatiza o fluxo completo de gerenciamento de vulnerabilidades em projetos com múltiplos ecossistemas. Com um único comando, ela é capaz de:

1. Varrer todos os lockfiles ( `composer.lock` , `package-lock.json` , `requirements.txt` , `Pipfile.lock` ) usando o [OSV Scanner](#)
2. Classificar as vulnerabilidades como seguras para atualizar ou como atualizações que precisam de autorização manual
3. Aplicar atualizações de patch e minor dentro de containers Docker isolados — não é necessário ter PHP, Node.js ou Python instalados localmente
4. Executar os comandos de validação (suíte de testes) dentro do mesmo container para confirmar que nada quebrou
5. Reverter todas as alterações automaticamente caso a validação falhe
6. Gerar um relatório executivo em HTML com comparação de vulnerabilidades antes e depois das correções
7. Fazer upload do relatório para o Google Drive
8. Abrir um pull request no GitHub com as mudanças seguras já validadas

Mudanças disruptivas (bumps de versão major, alterações de constraint) nunca são aplicadas automaticamente. Elas exigem autorização explícita por ecossistema via `--authorize-breaking`.

## Requisitos

| Ferramenta | Versão mínima    |
|------------|------------------|
| Node.js    | ≥ 24.0.0         |
| Docker     | qualquer recente |

Docker é o único requisito de runtime além do Node.js. OSV Scanner, SonarQube, npm, PHP Composer e pip rodam todos dentro de containers Docker efêmeros. Não é necessário instalar nenhuma dessas ferramentas localmente.

O `gh` CLI é necessário apenas se você usar `--open-pr`. Instale em [cli.github.com](https://cli.github.com).

## Instalação

Instalação global com npm:

```
npm install -g security-scan
```

Verificar a instalação:

```
security-scan --version
# security-scan/0.1.3
```

Executar sem instalar (útil para varreduras pontuais):

```
npx security-scan --help
```

## Início Rápido

### Passo 1: Gerar o arquivo de configuração

```
security-scan init
```

Isso inicia um assistente interativo que detecta seus ecossistemas (npm, composer, pip), solicita que você confirme ou ajuste a configuração e grava um `project-config.yml` no diretório atual.

Passo 2: Varrer por vulnerabilidades

```
security-scan scan
```

Exibe um resumo de todas as vulnerabilidades encontradas. Nenhum arquivo é modificado.

Passo 3: Aplicar correções seguras

```
security-scan fix
```

Executa o pipeline completo: scan → aplicar atualizações seguras → validar → reverter se quebrar → gerar relatório executivo.

Passo 4: Aplicar correções seguras e abrir um PR

```
security-scan fix --open-pr
```

Igual ao passo anterior, mas também cria uma branch no git, faz commit das mudanças, faz push e abre um pull request no GitHub.

Comandos

init

Gera um template de project-config.yml para o projeto atual.

```
security-scan init [options]
```

| Opção                 | Tipo    | Padrão               | Descrição                                           |
|-----------------------|---------|----------------------|-----------------------------------------------------|
| --project-name <name> | string  | prompt interativo    | Nome do projeto gravado na configuração             |
| --client <name>       | string  | prompt interativo    | Nome do cliente gravado na configuração             |
| --cwd <path>          | string  | diretório atual      | Diretório de trabalho para detecção de ecossistemas |
| --output <path>       | string  | ./project-config.yml | Caminho do arquivo de saída                         |
| --force               | boolean | false                | Sobrescrever o arquivo se ele já existir            |

O que acontece durante o init :

- 1. Verifica se project-config.yml já existe (falha a menos que --force esteja ativo).
- 2. Solicita o nome do projeto e do cliente (ou usa as flags da CLI).

3. Detecta o ambiente de runtime lendo arquivos do projeto:
  - **npm**: lê `.nvmrc` , `.node-version` , `package.json#engines.node`
  - **composer**: lê `.php-version` , `composer.json#require.php`
  - **pip**: lê `runtime.txt` , `.python-version`
4. Apresenta um seletor de ecossistemas com checkbox (os ecossistemas detectados vêm pré-selecionados).
5. Para cada ecossistema, solicita:
  - Estratégia de fix ( `osv` , `npm-audit` , `osv-then-audit` )
  - Comandos de validação (ex.: `npm test` , `php artisan test` )
  - Comandos de advisor (ex.: `npm audit --json` )
  - Versão do runtime (inferida ou digitada manualmente)
  - Estratégia de imagem ( `pull` ou `dockerfile` )
6. Pergunta se deve ativar a integração com SonarQube.
7. Pergunta o idioma dos relatórios ( `en` ou `pt-br` ).
8. Pergunta se deve gerar relatórios Markdown e onde salvá-los.
9. Grava o `project-config.yml` gerado.
10. Se SonarQube estiver ativo e `sonar-project.properties` não existir, cria um template inicial.

**Exemplo — modo não interativo (amigável para CI):**

```
security-scan init \  
  --project-name "Meu App" \  
  --client "Acme Corp" \  
  --force
```

No modo não interativo (quando stdin não é um TTY), o `init` seleciona automaticamente todos os ecossistemas detectados e seus valores padrão.

**Códigos de saída:** `0` sucesso, `3` erro de configuração ou de saída.

**scan**

Executa apenas a varredura de vulnerabilidades. Nenhum arquivo é modificado.

```
security-scan scan [options]
```

| Opção                                  | Tipo    | Padrão                            | Descrição                                       |
|----------------------------------------|---------|-----------------------------------|-------------------------------------------------|
| <code>-c, --config &lt;path&gt;</code> | string  | <code>./project-config.yml</code> | Caminho para o arquivo de configuração          |
| <code>--cwd &lt;path&gt;</code>        | string  | diretório atual                   | Diretório de trabalho (raiz do projeto)         |
| <code>--dry-run</code>                 | boolean | false                             | Exibir o que seria executado, sem executar nada |
| <code>-v, --verbose</code>             | boolean | false                             | Ativar saída verbosa                            |

| Opção               | Tipo    | Padrão | Descrição                                            |
|---------------------|---------|--------|------------------------------------------------------|
| -q, --quiet         | boolean | false  | Suprimir toda saída exceto erros e o relatório final |
| --json              | boolean | false  | Exibir resultado em JSON no stdout                   |
| -o, --output <path> | string  | stdout | Gravar saída em um arquivo                           |

O que acontece durante o scan :

1. Carrega e valida o project-config.yml usando o schema Zod. Sai com código 3 em caso de erro de validação.
2. Executa o osv-scanner dentro de um container Docker efêmero contra todos os lockfiles detectados no diretório de trabalho.
3. Analisa a saída do OSV e classifica cada resultado:
  - auto\_safe — atualização de patch/minor dentro das constraints atuais
  - breaking — requer bump de versão major ou mudança de constraint
4. Formata e emite o resultado (resumo em texto ou JSON).

Exemplos:

```
# Varredura básica
security-scan scan

# Varrer um projeto em outro diretório
security-scan scan --cwd /caminho/para/o/projeto

# Salvar resultados em JSON (útil como artefato de CI)
security-scan scan --json --output scan-results.json

# Modo silencioso: exibir apenas o resumo final
security-scan scan --quiet
```

Exemplo de saída:

```
security-scan scan summary
=====
npm          2 vulnerabilities (1 auto-safe, 1 breaking)
composer    0 vulnerabilities
pip          1 vulnerability (1 auto-safe)

Exit code: 1 (breaking vulnerabilities found)
```

Códigos de saída:

| Código | Significado                        |
|--------|------------------------------------|
| 0      | Nenhuma vulnerabilidade encontrada |

| Código | Significado                                    |
|--------|------------------------------------------------|
| 1      | Vulnerabilidades disruptivas encontradas       |
| 2      | Erro no scanner (falha no gate ou erro do OSV) |
| 3      | Erro de configuração                           |

fix

Pipeline completo: scan → aplicar atualizações seguras por ecossistema → validar → reverter se quebrar → gerar relatório executivo.

```
security-scan fix [options]
```

| Opção                        | Tipo     | Padrão               | Descrição                                                                                                            |
|------------------------------|----------|----------------------|----------------------------------------------------------------------------------------------------------------------|
| -c, --config <path>          | string   | ./project-config.yml | Caminho para o arquivo de configuração                                                                               |
| --cwd <path>                 | string   | diretório atual      | Diretório de trabalho (raiz do projeto)                                                                              |
| --phases <phases>            | string   | todas as fases       | Lista de fases separadas por vírgula. Valores aceitos: scan , npm , composer , pip , report                          |
| --no-report                  | boolean  | false                | Não gerar o relatório executivo                                                                                      |
| --authorize-breaking <id...> | string[] | nenhum               | Autorizar atualizações disruptivas para os ecossistemas especificados. Exemplo:<br>--authorize-breaking composer npm |
| --dry-run                    | boolean  | false                | Registrar as mudanças planejadas sem executar nada                                                                   |
| -v, --verbose                | boolean  | false                | Ativar saída verbosa                                                                                                 |
| -q, --quiet                  | boolean  | false                | Suprimir toda saída exceto erros e o relatório final                                                                 |
| --json                       | boolean  | false                | Exibir resultado em JSON                                                                                             |
| -o, --output <path>          | string   | stdout               | Gravar relatório em arquivo                                                                                          |
| --create-branch              | boolean  | false                | Criar uma branch git antes de aplicar as correções e fazer commit em caso de sucesso                                 |

| Opção                                       | Tipo    | Padrão                          | Descrição                                                                                                                      |
|---------------------------------------------|---------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>--branch-prefix &lt;prefix&gt;</code> | string  | <code>fix/security-scan-</code> | Prefixo do nome da branch                                                                                                      |
| <code>--open-pr</code>                      | boolean | <code>false</code>              | Criar um pull request no GitHub após o <code>fix</code> (implica <code>--create-branch</code> ; requer o CLI <code>gh</code> ) |
| <code>--pr-title &lt;title&gt;</code>       | string  | gerado automaticamente          | Título do pull request                                                                                                         |

### Fases do pipeline:

O comando `fix` executa as seguintes fases em ordem:

1. **scan** — executa o OSV Scanner como Gate A; classifica as vulnerabilidades.
2. **npm** — atualiza pacotes npm (se o ecossistema npm estiver configurado).
3. **composer** — atualiza pacotes PHP (se o ecossistema composer estiver configurado).
4. **pip** — atualiza pacotes Python (se o ecossistema pip estiver configurado).
5. **report** — gera o relatório executivo em HTML.

Use `--phases` para executar apenas um subconjunto:

```
# Executar somente as fases scan e npm
security-scan fix --phases scan,npm

# Executar todas as fases exceto o relatório
security-scan fix --no-report
```

### Autorizando mudanças disruptivas:

```
# Permitir que pacotes do composer sejam atualizados para versões disruptivas
security-scan fix --authorize-breaking composer

# Permitir atualizações disruptivas em npm e composer
security-scan fix --authorize-breaking npm composer
```

A autorização é por execução e nunca é persistida no arquivo de configuração.

### Variável de ambiente kill-switch:

```
# Pular todas as correções automáticas após a fase de scan
SECURITY_SCAN_NO_AUTO_FIX=1 security-scan fix
```

Útil em pipelines de CI onde você quer o resultado do scan registrado, mas sem mutações em arquivos.

### Fluxo de branch e PR:



```
# Criar branch, aplicar correções e fazer commit em caso de sucesso
security-scan fix --create-branch

# Criar branch E abrir um PR no GitHub
security-scan fix --open-pr

# Prefixo de branch personalizado
security-scan fix --create-branch --branch-prefix deps/security-fix-

# Título de PR personalizado
security-scan fix --open-pr --pr-title "chore: security dependency updates"
```

**Códigos de saída:**

| Código | Significado                                                                                |
|--------|--------------------------------------------------------------------------------------------|
| 0      | Tudo resolvido (ou nada a corrigir)                                                        |
| 1      | Vulnerabilidades encontradas / erros de atualização / vulnerabilidades pendentes restantes |
| 2      | Falha no gate ou erro no scanner                                                           |
| 3      | Erro de configuração                                                                       |

**Detalhe do pipeline por ecossistema:**

Para cada plugin de ecossistema:

- 1. Executa advisors (apenas informativos — nunca bloqueiam o pipeline).
- 2. Pula o plugin se não houver vulnerabilidades `auto_safe` (e nenhuma `breaking` com `--authorize-breaking`).
- 3. Resolve o runner de container Docker (npm/pip/composer).
- 4. Para npm: auto-rebaixa a estratégia `osv` / `osv-then-audit` para `npm-audit` se o `package-lock.json` tiver `lockfileVersion: 1` (o `osv-scanner` não consegue corrigir lockfiles v1 in-place).
- 5. Chama o updater do plugin.
- 6. Opcionalmente instala pacotes disruptivos ( `--authorize-breaking` ).
- 7. Executa verificação residual do OSV pós-atualização para confirmar que as correções foram aplicadas.
- 8. Valida o resultado da atualização contra o gate do ecossistema (schema Zod).

Em caso de sucesso: aplica as atualizações e executa os comandos de validação.

Em caso de falha na validação: reverte todas as mudanças naquele ecossistema e continua com os demais.

**executive-report**

Gera um relatório executivo em HTML a partir dos últimos resultados de varredura.

```
security-scan executive-report [options]
```

| Opção               | Tipo    | Padrão                  | Descrição                                                    |
|---------------------|---------|-------------------------|--------------------------------------------------------------|
| -c, --config <path> | string  | ./project-config.yml    | Caminho para o arquivo de configuração                       |
| --cwd <path>        | string  | diretório atual         | Diretório de trabalho                                        |
| --client <name>     | string  | da configuração         | Nome do cliente (sobrescreve project.client na configuração) |
| --project <name>    | string  | da configuração         | Nome do projeto (sobrescreve project.name na configuração)   |
| -o, --output <path> | string  | diretório de relatórios | Gravar relatório em arquivo                                  |
| --dry-run           | boolean | false                   | Exibir comandos sem executar                                 |
| -v, --verbose       | boolean | false                   | Ativar saída verbosa                                         |
| -q, --quiet         | boolean | false                   | Suprimir toda saída exceto erros e o relatório final         |
| --json              | boolean | false                   | Exibir resultado em JSON                                     |

O que acontece:

- 1. Executa uma varredura de vulnerabilidades atualizada (estado antes).
- 2. Executa o pipeline completo do orquestrador.
- 3. Renderiza o relatório executivo em HTML usando templates Handlebars.
- 4. Salva o relatório no diretório de saída configurado.
- 5. Opcionalmente faz upload para o Google Drive se cloud\_storage estiver configurado.

O idioma do relatório é controlado por report\_language no project-config.yml ( en ou pt-br ).

Exemplo:

```
# Gerar relatório com nome de cliente personalizado
security-scan executive-report --client "Acme Corp" --output relatorio.html
```

cloud-setup

Seletor interativo de pastas do Google Drive. Salva o ID da pasta escolhida no project-config.yml para que futuras execuções de fix e executive-report façam upload automático dos relatórios.

```
security-scan cloud-setup [options]
```

| Opção               | Tipo   | Padrão               | Descrição                              |
|---------------------|--------|----------------------|----------------------------------------|
| -c, --config <path> | string | ./project-config.yml | Caminho para o arquivo de configuração |

| Opção                           | Tipo   | Padrão          | Descrição             |
|---------------------------------|--------|-----------------|-----------------------|
| <code>--cwd &lt;path&gt;</code> | string | diretório atual | Diretório de trabalho |

### Pré-requisitos:

As credenciais do Google OAuth precisam estar disponíveis. A CLI lê as seguintes variáveis de ambiente:

- `G00GLE_CLIENT_ID` — seu client ID do OAuth 2.0
- `G00GLE_CLIENT_SECRET` — seu client secret do OAuth 2.0

Para obter as credenciais:

1. Acesse o [Google Cloud Console](#)
2. Crie um projeto (ou use um existente)
3. Ative a Google Drive API
4. Crie credenciais OAuth 2.0 (tipo Desktop app)
5. Copie o client ID e o secret para suas variáveis de ambiente

### O que acontece:

1. Verifica se há tokens OAuth armazenados (de uma execução anterior de `cloud-setup`).
2. Se ainda não autenticado, abre a URL de autorização do Google OAuth 2.0 no navegador usando `execFile` com `shell: false` (sem possibilidade de injeção de shell).
3. Após a autenticação, lista as pastas do seu Google Drive.
4. Apresenta um seletor interativo de pasta.
5. Grava o `folder_id` selecionado em `cloud_storage.google_drive.folder_id` no `project-config.yml`.

### Exemplo de fluxo:

```
# Configurar integração com Google Drive
security-scan cloud-setup

# Após a configuração, execuções do fix farão upload automaticamente
security-scan fix

# Para exigir sucesso no upload (falhar CI se o upload falhar)
# Defina no project-config.yml:
#   cloud_storage:
#     require_upload: true
```

## Referência de Configuração

O `project-config.yml` é a única fonte de verdade para todo o comportamento do `security-scan`. Abaixo está a referência completa e anotada de todos os campos.

## project

```
project:
  name: 'Meu Projeto'      # Obrigatório. Nome do projeto usado nos relatórios.
  client: 'Acme Corp'      # Obrigatório. Nome do cliente usado nos relatórios.
```

## report\_language

```
report_language: 'pt-br'   # 'en' | 'pt-br' (padrão: 'en')
```

Controla o locale dos relatórios executivos gerados. Afeta todo o texto dos relatórios HTML e Markdown. Não afeta a saída da CLI.

## config\_version

```
config_version: '1'       # Opcional. Para detecção de compatibilidade futura.
```

## ecosystems

Lista declarativa de ecossistemas a varrer e atualizar. Pelo menos uma entrada é obrigatória.

```
ecosystems:
- id: 'npm'
  fixer: 'osv-then-audit'      # osv | npm-audit | osv-then-audit
  validationCommands:
    - name: 'Tests'
      command: 'npm test'
      timeout_seconds: 120      # opcional; padrão: 300 (5 minutos)
  advisors:
    - name: 'audit'
      command: 'npm audit --json'
      format: 'json'           # json | text (padrão: text)

- id: 'composer'
  fixer: 'osv'
  validationCommands:
    - name: 'Tests'
      command: 'php artisan test'
      timeout_seconds: 300
  advisors: []

- id: 'pip'
  fixer: 'osv'
  validationCommands:
    - name: 'Tests'
      command: 'pytest'
```

**Campos do ecossistema:**

| Campo                                | Tipo                 | Obrigatório | Descrição                                                                            |
|--------------------------------------|----------------------|-------------|--------------------------------------------------------------------------------------|
| id                                   | npm   composer   pip | Sim         | Identificador do ecossistema                                                         |
| fixer                                | string               | Não         | Estratégia de fix (veja <a href="#">Estratégias de Fix</a> )                         |
| validationCommands                   | array                | Não         | Comandos executados após as atualizações para verificar que nada quebrou             |
| validationCommands[].name            | string               | Sim         | Rótulo legível para o comando                                                        |
| validationCommands[].command         | string               | Sim         | String de comando shell (executado dentro do container Docker)                       |
| validationCommands[].timeout_seconds | number               | Não         | Timeout em segundos; padrão: 300                                                     |
| advisors                             | array                | Não         | Comandos informativos executados antes das atualizações (nunca bloqueiam o pipeline) |
| advisors[].name                      | string               | Sim         | Rótulo legível                                                                       |
| advisors[].command                   | string               | Sim         | String de comando shell                                                              |
| advisors[].format                    | json   text          | Não         | Formato de saída; use json para npm audit --json                                     |

**Nota de segurança sobre validationCommands :** Esses comandos são executados dentro do container Docker do ecossistema via `sh -c` . Não estão expostos a entrada externa — apenas comandos criados no `project-config.yml` (que você controla) são executados. Comandos que começam com `git` , `gh` ou `open` são exceções e executam no host.

**protected\_packages**

Pacotes listados aqui nunca são atualizados além da constraint declarada. Qualquer atualização que exija mudança de constraint requer `--authorize-breaking` explícito.

```
protected_packages:
  npm:
    - package: 'tailwindcss'
      constraint: '^3.3.3'
      reason: 'Tailwind v4 tem mudanças disruptivas de config e requer migração'
    - package: 'react'
      constraint: '^18.0.0'
      reason: 'Migração para React 19 requer ciclo completo de QA'
  composer:
    - package: 'laravel/framework'
      constraint: '^10.8'
      reason: 'Upgrade major para Laravel 11 requer um projeto dedicado'
  pip:
    - package: 'django'
      constraint: '>=4.2,<5.0'
      reason: 'Django 5.x tem mudanças disruptivas'
```

**Campos por entrada:**

| Campo      | Tipo   | Obrigatório | Descrição                                        |
|------------|--------|-------------|--------------------------------------------------|
| package    | string | Sim         | Nome do pacote como aparece no lockfile          |
| constraint | string | Sim         | A constraint de versão que não deve ser excedida |
| reason     | string | Sim         | Motivo legível (aparece nos relatórios)          |

**safe\_update\_policy**

```
safe_update_policy:
  allow_patch_and_minor_within_constraints: true # padrão: true
  require_authorization_for_constraint_change: true # padrão: true
```

| Campo                                       | Padrão | Descrição                                                                                                     |
|---------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------|
| allow_patch_and_minor_within_constraints    | true   | Aplicar automaticamente atualizações de patch e minor que permaneçam dentro das constraints ^ / ~ / >= atuais |
| require_authorization_for_constraint_change | true   | Exigir --authorize-breaking para qualquer atualização que mude a constraint de versão declarada               |

**conflict\_resolution**

```
conflict_resolution: 'manual' # atualmente apenas 'manual' é suportado
```

# scanners

Controla quais engines de scanning são usadas e como são configuradas.

```
scanners:
  primary: 'osv'          # Engine usada como fonte do Gate A. Padrão: 'osv'
  osv:
    runner: 'docker'      # docker (padrão) | local | auto
    image: 'ghcr.io/google/osv-scanner:latest'  # opcional; padrão mostrado
    args: []              # opcional: args adicionais da CLI repassados ao osv-scanner
  sonarqube:
    enabled: false        # defina true para ativar a integração com SonarQube
    mode: 'external'      # external (padrão) | managed
    on_failure: 'warn'    # warn (padrão) | fail
    # modo external: lê de sonar-project.properties; variável SONAR_TOKEN fornece a autenticação.
    # modo managed: a CLI provisiona um container SonarQube CE efêmero, gera um token
    #                      e o derruba após o scan.
    scanner_image: 'sonarsource/sonar-scanner-cli:latest'  # opcional
    server_image: 'sonarqube:lts-community'                # opcional (apenas modo managed)
    send_branch_name: false    # apenas Developer/Enterprise Edition; false = seguro para CE
    ce_task_timeout_seconds: 120 # segundos para aguardar a conclusão da tarefa CE
    scanner_timeout_seconds: 300 # segundos antes de matar o processo do sonar-scanner
    dynamic_timeout: true      # escalar timeouts com base no ncloc da análise anterior
    timeout_scale:
      scanner_seconds_per_kloc: 3 # segundos de budget do scanner por 1000 linhas
      ce_seconds_per_kloc: 1.5    # segundos de budget do CE por 1000 linhas
    scanner_jvm_opts: '-Xmx2048m' # opcional; aumentar heap para codebases grandes
```

## Modos do runner OSV:

| Modo   | Comportamento                                                                                            |
|--------|----------------------------------------------------------------------------------------------------------|
| docker | Sempre executar o osv-scanner via container Docker efêmero. <b>Padrão e recomendado.</b>                 |
| local  | Usar o binário osv-scanner instalado localmente. Falha se não estiver instalado. Emite aviso.            |
| auto   | Tentar local primeiro; recorrer ao Docker se indisponível. <b>Modo de escape obsoleto — emite aviso.</b> |

# runners

Configuração de container por ecossistema. Controla qual imagem Docker é usada, a versão do runtime e dependências opcionais de SO.

```

runners:
  npm:
    mode: 'docker'          # docker (padrão) | local | auto
    language_version: '20'  # inferido de .nvmrc / package.json se ausente
    image: 'node:20'        # override explícito; tem precedência sobre language_version
    image_source: 'pull'    # pull (padrão) | dockerfile
    dockerfile_path: './Dockerfile' # obrigatório quando image_source='dockerfile'
    build_context: '.'      # padrão: raiz do projeto
    build_args:             # passados como --build-arg KEY=VALUE ao docker build
      NODE_VERSION: '20'
    native_deps:            # pacotes de SO para instalar com apt-get antes dos comandos npm
      - libvips-dev         # necessário para sharp@0.x
      - build-essential     # necessário para addons nativos que usam node-gyp
      - python3             # necessário para node-gyp em algumas distros
    allow_build_context_escape: false # segurança: permitir contexto fora da raiz do projeto

  composer:
    mode: 'docker'
    language_version: '8.1' # inferido de .php-version / composer.json se ausente
    image: 'php:8.1-cli'    # override explícito
    image_source: 'pull'    # pull | dockerfile
    dockerfile_path: './Dockerfile'
    build_context: '.'
    build_args: {}
    ignore_platform_reqs: true # padrão true no modo docker; passa --ignore-platform-reqs
    native_deps:
      - imagemagick
      - libmagickwand-dev

  pip:
    mode: 'docker'
    language_version: '3.11' # inferido de runtime.txt / .python-version se ausente
    image: 'python:3.11-slim' # override explícito
    image_source: 'pull'     # pull | dockerfile
    dockerfile_path: './Dockerfile'
    build_context: '.'
    build_args: {}
    native_deps:
      - libjpeg-dev          # necessário para Pillow
      - libpq-dev            # necessário para psycopg2

```

**Opções de modo do runner (iguais para npm, composer e pip):**

| Modo   | Comportamento                                                                |
|--------|------------------------------------------------------------------------------|
| docker | Executar dentro de um container Docker efêmero. <b>Padrão e recomendado.</b> |
| local  | Usar o binário instalado localmente. Emite aviso.                            |
| auto   | Tentar local primeiro; recorrer ao Docker. <b>Obsoleto — emite aviso.</b>    |



## scan (caminhos de varredura)

Controla quais caminhos o `osv-scanner` inspeciona.

```
scan:
  auto_discover: true      # padrão: true; também varrer raiz do projeto em busca de lockfiles
  paths:                  # caminhos explícitos para varrer
    - 'frontend/'         # diretórios (barra final) são varridos recursivamente via -r
    - 'backend/package-lock.json' # caminhos explícitos de arquivo usam --lockfile
  exclude:                # caminhos a excluir
    - 'vendor/'
    - 'node_modules/'
```

**Restrições sobre paths:** Todas as entradas devem ser relativas (sem `/` inicial) e não devem conter segmentos `..` ou caracteres glob. Os caminhos se resolvem em relação a `/project` dentro do container.

## outputs

Controla o local e os formatos dos relatórios.

```
outputs:
  dir: './reports'          # diretório de saída; padrão: .security-scan/reports
  sub_folders: false        # quando true, relatórios de engine vão para sub-pastas (sonarqube/)
  formats:
    - 'markdown'            # HTML sempre é gerado; markdown é opcional
```

O relatório executivo em HTML sempre é gerado. O Markdown só é gerado quando `markdown` está incluído em `formats`.

## cloud\_storage

Configura o upload automático de relatórios para o Google Drive após cada execução de `fix` ou `executive-report`.

```
cloud_storage:
  provider: 'google_drive'  # apenas google_drive é suportado
  google_drive:
    folder_id: 'SEU_FOLDER_ID' # definido pelo comando cloud-setup
    require_upload: false      # se true, sai com código 1 quando o upload falha
```

Execute `security-scan cloud-setup` para autenticar e selecionar a pasta de forma interativa.

## workflow

Configuração do fluxo de branch e PR no git. As flags da CLI sempre sobrescrevem esses valores.

```
workflow:
  create_branch: false          # criar branch git antes de aplicar correções
  open_pr: false                # fazer push da branch e abrir PR no GitHub em caso de sucesso
  branch_prefix: 'fix/security-scan-' # prefixo para nomes de branch gerados automaticamente
  pr_title: ''                  # título personalizado do PR; gerado automaticamente se ausente
```

As flags da CLI ( `--create-branch` , `--open-pr` , `--branch-prefix` , `--pr-title` ) têm precedência sobre esses valores por execução.

## Docker e Estratégias de Runtime

Todos os CLIs de ecossistema (npm, composer, pip) e os scanners (osv-scanner) executam dentro de containers Docker efêmeros por padrão. Isso significa:

- Não é necessário ter Node.js, PHP ou Python instalados localmente além do próprio CLI do security-scan.
- Cada execução obtém um ambiente limpo e isolado.
- As versões dos containers correspondem ao runtime declarado do projeto (inferido ou configurado).
- Os containers são removidos automaticamente após cada execução ( `--rm` ).

## Image Source: pull vs dockerfile

Cada runner suporta duas estratégias de imagem:

**pull (padrão):** Baixar uma imagem pré-construída do Docker Hub ou outro registry.

```
runners:
  npm:
    image_source: 'pull'
    language_version: '20' # resolve para node:20
```

**dockerfile :** Construir uma imagem local a partir de um Dockerfile do próprio projeto. Use quando o projeto tem dependências de sistema não padrão ou uma imagem base personalizada.

```
runners:
  npm:
    image_source: 'dockerfile'
    dockerfile_path: '.docker/node.Dockerfile'
    build_context: '.'
    build_args:
      NODE_VERSION: '20'
      APP_ENV: 'production'
```

A estratégia `dockerfile` é mutuamente exclusiva com o campo `image` . Quando

`allow_build_context_escape: true` , o contexto de build pode alcançar fora da raiz do projeto — isso emite um aviso porque envia uma árvore de diretórios maior para o daemon Docker.

# Resolução de Versão do Runtime

Quando `image` não está definido, o runner resolve a imagem Docker a partir da versão do runtime usando esta precedência:

## npm:

1. `runners.npm.language_version` da configuração (ex.: `'20' → node:20`)
2. Inferido de `.nvmrc` / `.node-version` / `package.json#engines.node`
3. Recorre a `node:lts`

## composer:

1. `runners.composer.language_version` da configuração (ex.: `'8.2' → php:8.2-cli`)
2. Inferido de `.php-version` / `composer.json#require.php`
3. Recorre a `composer:2`

## pip:

1. `runners.pip.language_version` da configuração (ex.: `'3.11' → python:3.11-slim`)
2. Inferido de `runtime.txt` / `.python-version`
3. Recorre a `python:3-slim`

# Dependências Nativas de SO

Alguns pacotes npm (ex.: `sharp`, `canvas`) ou extensões PHP (ex.: `imagick`) requerem bibliotecas de SO para compilar. Use `native_deps` para instalá-las via `apt-get` dentro do container efêmero:

```
runners:
  npm:
    native_deps:
      - libvips-dev          # necessário para sharp
      - build-essential     # necessário para qualquer add-on nativo que usa node-gyp
      - python3              # necessário para node-gyp em algumas distros
  composer:
    native_deps:
      - imagemagick
      - libmagickwand-dev
  pip:
    native_deps:
      - libjpeg-dev         # Pillow
      - libpq-dev           # psycopg2
```

Os pacotes são instalados com `apt-get install -y --no-install-recommends` antes do CLI do ecossistema executar. Os nomes de pacotes devem seguir as convenções de nomenclatura Debian (alfanumérico minúsculo, hífen, pontos e sinais de adição apenas).

# Engines de Scanner

## OSV Scanner

A engine de scanning primária. O OSV Scanner usa o banco de dados [Open Source Vulnerabilities](#) do Google para encontrar vulnerabilidades conhecidas em lockfiles.

### Lockfiles suportados:

- `package-lock.json` (npm)
- `yarn.lock` (npm, apenas leitura — atualizações via npm)
- `composer.lock` (PHP Composer)
- `requirements.txt` , `Pipfile.lock` (Python pip)

### Configuração:

```
scanners:
  primary: 'osv'      # OSV é a fonte padrão do Gate A
  osv:
    runner: 'docker'
    image: 'ghcr.io/google/osv-scanner:latest'
    args:
      - '--experimental-call-analysis'  # flags extras opcionais
```

O OSV Scanner executa em um container Docker efêmero. O diretório do projeto é montado como somente leitura dentro do container. Nenhum lockfile é modificado durante a fase de scan.

## SonarQube

Uma engine de scanning secundária opcional para análise de qualidade de código.

### Modo external (padrão quando ativado):

Usa uma instância SonarQube pré-existente. A configuração vem de `sonar-project.properties` na raiz do projeto. A autenticação usa a variável de ambiente `SONAR_TOKEN`.

```
scanners:
  sonarqube:
    enabled: true
    mode: 'external'
    on_failure: 'warn'  # warn | fail
```

Crie o arquivo `sonar-project.properties`:

```
sonar.projectKey=meu-projeto
sonar.projectName=Meu Projeto
sonar.sources=src
sonar.exclusions=**/node_modules/**,**/vendor/**
sonar.host.url=https://sonarqube.exemplo.com
```

Defina o token de autenticação:

```
export SONAR_TOKEN=seu_token_aqui
```

### Modo managed:

A CLI provisiona um container SonarQube Community Edition efêmero, executa o scan e depois o derruba.

```
scanners:
  sonarqube:
    enabled: true
    mode: 'managed'
    server_image: 'sonarqube:lts-community'
    scanner_image: 'sonarsource/sonar-scanner-cli:latest'
    on_failure: 'warn'
```

Nota: `send_branch_name: true` requer SonarQube Developer Edition ou superior. Community Edition não suporta análise de branches.

### Resultados do SonarQube nos relatórios:

Quando o SonarQube está ativo, o relatório executivo inclui:

- Status do Quality Gate (PASSED / FAILED)
- Condições do Quality Gate
- Métricas: bugs, vulnerabilidades, code smells, cobertura, linhas duplicadas, NCLOC
- Issues por arquivo

## Plugins de Ecossistema e Estratégias de Fix

### npm

Varre `package-lock.json` e aplica atualizações de dependências npm.

### Estratégias de fix:

| Estratégia     | Comportamento                                                                                                                                                                                                                    |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| osv            | O OSV Scanner aplica correções in-place ao <code>package-lock.json</code> . Mudanças disruptivas são aplicadas separadamente pelo npm via <code>npm install &lt;pkg&gt;@&lt;version&gt;</code> .                                 |
| npm-audit      | Usa <code>npm audit fix</code> exclusivamente. O fix do OSV não é executado neste caminho.                                                                                                                                       |
| osv-then-audit | Aplica o fix do OSV primeiro, depois executa <code>npm audit fix</code> em cima. Se a validação falhar após ambos, reverte a porção do <code>npm-audit</code> e revalida contra o estado somente-OSV.<br><b>Padrão para npm.</b> |

#### Auto-rebaixamento:

Se o `package-lock.json` tiver `lockfileVersion: 1` (`npm ≤ 6`), as estratégias `osv` e `osv-then-audit` são automaticamente rebaixadas para `npm-audit` porque o osv-scanner não consegue corrigir lockfiles v1 in-place. Esse rebaixamento é registrado como aviso.

## composer

Varre `composer.lock` e aplica atualizações de pacotes PHP usando o Composer.

#### Estratégia de fix:

| Estratégia | Comportamento                                                                                                                         |
|------------|---------------------------------------------------------------------------------------------------------------------------------------|
| osv        | O OSV Scanner identifica pacotes vulneráveis; o Composer é usado para atualizá-los. <b>Única estratégia disponível para composer.</b> |

**Imagem padrão:** `php:<versão>-cli` (ex.: `php:8.2-cli`)

**Requisitos de plataforma:** `ignore_platform_reqs: true` é definido por padrão no modo Docker porque o container não é o ambiente de produção — verificações de extensões PHP contra o build do PHP do container são irrelevantes.

## pip

Varre `requirements.txt` ou `Pipfile.lock` e aplica atualizações de pacotes Python usando pip.

#### Estratégia de fix:

| Estratégia | Comportamento                                                                                                               |
|------------|-----------------------------------------------------------------------------------------------------------------------------|
| osv        | O OSV Scanner identifica pacotes vulneráveis; o pip é usado para atualizá-los. <b>Única estratégia disponível para pip.</b> |

**Imagem padrão:** `python:<versão>-slim` (ex.: `python:3.11-slim`)

# Estratégias de Fix

| Estratégia     | Ecossistemas       | Descrição                                                                                                                                                                                                |
|----------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| osv            | npm, composer, pip | O OSV Scanner realiza correções in-place nos lockfiles. É o método mais preciso — as correções são retiradas diretamente do banco de dados OSV.                                                          |
| npm-audit      | apenas npm         | Delega o fix ao <code>npm audit fix</code> . Mais rápido, mas menos preciso que o OSV para árvores de dependência complexas.                                                                             |
| osv-then-audit | apenas npm         | Aplica o fix do OSV primeiro para precisão, depois executa <code>npm audit fix</code> para capturar problemas restantes. Recua graciosamente para somente-OSV se o audit-fix causar falhas na validação. |

## Pacotes Protegidos e Política de Atualização Segura

Os mecanismos de pacotes protegidos e de política de atualização segura trabalham juntos para prevenir mudanças disruptivas acidentais.

### Como a Proteção Funciona

- Quando uma vulnerabilidade é encontrada em um pacote protegido:
  - Se o fix permanece dentro da `constraint` declarada, é classificado como `auto_safe` e aplicado normalmente.
  - Se o fix requer exceder a `constraint` (ex.: `^3.x`  $\rightarrow$  `^4.x`), é classificado como `breaking` e ignorado.
- Vulnerabilidades `breaking` são reportadas no relatório executivo com o motivo de por que não foram corrigidas.
- Para aplicar uma atualização disruptiva a um pacote protegido:

```
security-scan fix --authorize-breaking npm
```

Isso autoriza todas as atualizações disruptivas para npm nesta execução. A autorização não é persistida.

## Regras da Política de Atualização Segura

```
safe_update_policy:
  allow_patch_and_minor_within_constraints: true
  require_authorization_for_constraint_change: true
```

Com os padrões acima:

- `lodash@4.17.19`  $\rightarrow$  `lodash@4.17.21` (patch dentro de `^4.17.0`)  $\rightarrow$  **aplicado automaticamente**
- `lodash@4.17.21`  $\rightarrow$  `lodash@5.0.0` (bump major, mudança de constraint necessária)  $\rightarrow$  **bloqueado, autorização obrigatória**

# Fluxo de Branch e PR no Git

Por padrão, `security-scan fix` muda a árvore de trabalho diretamente (in-place). Use `--create-branch` para encapsular o fix em uma branch git revisável.

## Ciclo de Vida da Branch

```
security-scan fix --create-branch
```

1. Detecta a branch git atual.
2. Cria uma nova branch: `fix/security-scan-<ISO-timestamp>` (ex.: `fix/security-scan-2026-05-06T14:30:00.000Z`).
3. Executa o pipeline completo de fix na nova branch.
4. Em caso de sucesso: faz stage de todas as mudanças e commit com mensagem:  
`fix: apply safe dependency updates [security-scan]`
5. Em caso de falha: faz checkout da branch original e exclui a branch de fix. Nenhum commit é feito.

## Criação de PR

```
security-scan fix --open-pr
```

Implica `--create-branch`. Após um commit bem-sucedido:

1. Executa `git push origin <branch>`.
2. Executa `gh pr create` com título e corpo gerados automaticamente.

O corpo do PR inclui:

- Resumo do ecossistema (quais ecossistemas foram atualizados)
- Atribuição da versão do security-scan
- Co-authored with security-scan v<version>

**Pré-requisito:** CLI `gh` instalado e autenticado (`gh auth login`).

## Configuração Personalizada

```
# Prefixo de branch personalizado
security-scan fix --create-branch --branch-prefix deps/security-fix-

# Título de PR personalizado
security-scan fix --open-pr --pr-title "chore: atualizações de segurança de dependências"
```

Ou defina no `project-config.yml` (as flags da CLI sempre sobrescrevem):



```
workflow:
  create_branch: true
  open_pr: true
  branch_prefix: 'deps/security-'
  pr_title: 'chore: security dependency updates'
```

## Integração com CI/CD

### GitHub Actions — Apenas Scan

A integração de CI mais simples: varrer em um cronograma e em mudanças de lockfile.

```
# .github/workflows/security-scan.yml
name: Security scan

on:
  schedule:
    - cron: '0 6 * * 1' # Toda segunda-feira às 6h UTC
  push:
    paths:
      - 'composer.lock'
      - 'package-lock.json'
      - 'requirements.txt'
      - 'Pipfile.lock'

jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '24'

      - name: Instalar security-scan
        run: npm install -g security-scan

      - name: Executar varredura de vulnerabilidades
        run: security-scan scan --json --output scan-results.json

      - name: Upload dos resultados
        uses: actions/upload-artifact@v4
        with:
          name: scan-results
          path: scan-results.json
```

## GitHub Actions — Auto-fix com PR

Automação completa: scan, fix e abertura de PR quando vulnerabilidades são encontradas.

```
# .github/workflows/security-fix.yml
name: Security auto-fix

on:
  schedule:
    - cron: '0 6 * * 1' # Toda segunda-feira às 6h UTC

jobs:
  fix:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write

    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: actions/setup-node@v4
        with:
          node-version: '24'

      - name: Instalar security-scan
        run: npm install -g security-scan

      - name: Aplicar correções seguras e abrir PR
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: security-scan fix --open-pr
```

## GitHub Actions — Apenas Scan (Kill-switch)

Use o kill-switch para obter o resultado do scan em CI sem aplicar nenhuma correção:

```
- name: Scan (sem correções)
  run: SECURITY_SCAN_NO_AUTO_FIX=1 security-scan fix --json --output scan-results.json
```

## Tratamento de códigos de saída no CI

Os códigos de saída do security-scan se integram naturalmente com pipelines de CI:

```
# Falhar o pipeline se vulnerabilidades forem encontradas
security-scan scan
echo "Código de saída: $?"

# Permitir saída 1 (vulnerabilidades) mas falhar em erros de config (3)
security-scan scan || [ $? -le 1 ]
```

## Variáveis de Ambiente

| Variável                    | Efeito                                                                                                                                                                                                                                          |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SECURITY_SCAN_NO_AUTO_FIX=1 | Ignora todas as correções automatizadas após a fase de scan. O scan ainda é executado e o código de saída ainda reflete o status de vulnerabilidades. Útil em pipelines onde você quer o resultado do scan registrado sem mutações em arquivos. |
| NPM_DEFAULT_FIXER           | Sobrescreve a estratégia padrão de fix para npm. Valores válidos: osv , npm-audit , osv-then-audit . Padrão: osv-then-audit .                                                                                                                   |
| CLI_NAME                    | Sobrescreve o nome do binário da CLI usado na saída visível ao usuário e no nome da variável kill-switch. Padrão: deep-health . Quando definido como security-scan , o kill-switch passa a ser SECURITY_SCAN_NO_AUTO_FIX .                      |
| LOG_LEVEL=debug             | Ativa o logging no nível debug para saída interna detalhada.                                                                                                                                                                                    |
| SONAR_TOKEN                 | Token de autenticação para SonarQube no modo external . Obrigatório quando SonarQube está ativado com mode: external .                                                                                                                          |
| GOOGLE_CLIENT_ID            | Client ID OAuth 2.0 do Google. Obrigatório para cloud-setup e upload no Google Drive.                                                                                                                                                           |
| GOOGLE_CLIENT_SECRET        | Client secret OAuth 2.0 do Google. Obrigatório para cloud-setup e upload no Google Drive.                                                                                                                                                       |

## Códigos de Saída

Todos os comandos seguem a mesma convenção de códigos de saída:

| Código | Significado           | Quando ocorre                                                                                       |
|--------|-----------------------|-----------------------------------------------------------------------------------------------------|
| 0      | Limpo — sucesso       | Nenhuma vulnerabilidade encontrada, ou todas resolvidas                                             |
| 1      | Problemas encontrados | Vulnerabilidades encontradas, erros de atualização, ou vulnerabilidades pendentes restam após o fix |

| Código | Significado          | Quando ocorre                                                                           |
|--------|----------------------|-----------------------------------------------------------------------------------------|
| 2      | Erro no scanner/gate | Falha na validação do gate, erro do OSV, ou falha inesperada do scanner                 |
| 3      | Erro de configuração | project-config.yml não encontrado, schema inválido, ou erro de caminho de saída do init |

Esses códigos tornam o security-scan utilizável como gate em pipelines de CI/CD:

```
security-scan scan && echo "Limpo!" || echo "Problemas encontrados (código $?)"
```

## Solução de Problemas

### "security-scan requires Node.js >=24"

```
security-scan requires Node.js >=24. Detected: v20.x.x  
Please upgrade Node.js and try again.
```

Faça upgrade do Node.js para a versão 24 ou superior. Use [nvm](#) para gerenciamento fácil de versões:

```
nvm install 24  
nvm use 24
```

### "Config file not found"

```
Config file not found: ./project-config.yml  
Run "security-scan init" first.
```

Gere o arquivo de configuração:

```
security-scan init
```

Ou especifique o caminho explicitamente:

```
security-scan scan --config /caminho/para/project-config.yml
```

### Docker não disponível

```
Error: docker: command not found
```

Instale o Docker em [docs.docker.com](https://docs.docker.com) e verifique se o daemon Docker está em execução:

```
docker --version
docker ps
```

## "File already exists" durante o init

```
File already exists: ./project-config.yml
Use --force to overwrite.
```

Use `--force` para regenerar a configuração:

```
security-scan init --force
```

## SonarQube — "SONAR\_TOKEN not set"

```
SONAR_TOKEN environment variable is required for SonarQube external mode
```

Defina o token:

```
export SONAR_TOKEN=seu_token_aqui
security-scan scan
```

Ou adicione-o como secret no ambiente de CI.

## Vulnerabilidades disruptivas não corrigidas

Esse é o comportamento esperado. Vulnerabilidades classificadas como `breaking` requerem autorização explícita:

```
security-scan fix --authorize-breaking npm composer
```

Verifique a saída do scan para saber quais pacotes precisam de autorização.

## npm audit fix causa falha na validação

Ao usar a estratégia `osv-then-audit` e o `npm audit fix` quebrar a validação, o `security-scan` reverte automaticamente a porção do `npm audit fix` e revalida contra o estado somente-OSV. Se o estado somente-OSV também falhar na validação, todas as mudanças no npm são revertidas.

## CLI gh não encontrado para criação de PR

```
--open-pr requires the GitHub CLI (gh). Install it from https://cli.github.com and run: gh auth login
```

Instale o `gh` e autentique:

```
# macOS
brew install gh

# Linux
# Veja https://github.com/cli/cli/blob/trunk/docs/install_linux.md

gh auth login
```

## Upload para o Google Drive falha

Se `require_upload: false` (padrão), falhas de upload não são fatais — um aviso é exibido no `stderr`. Se `require_upload: true`, o comando sai com código `1`.

Execute `security-scan cloud-setup` para reautenticar se os tokens tiverem expirado.

## Comandos de validação atingem o timeout

Aumente o `timeout_seconds` para o comando de validação relevante:

```
ecosystems:
  - id: composer
    validationCommands:
      - name: 'Tests'
        command: 'php artisan test'
        timeout_seconds: 600 # aumentar do padrão de 300
```

## Perguntas Frequentes

### P: O `security-scan` modifica meus lockfiles diretamente?

Sim. Quando você executa `security-scan fix`, ele modifica `package-lock.json`, `composer.lock` e `requirements.txt` / `Pipfile.lock` dentro de containers Docker efêmeros. Use `--create-branch` para conter essas mudanças em uma branch revisável, ou `--dry-run` para ver o que aconteceria sem fazer alterações.

### P: O que acontece se minha suíte de testes falhar após uma atualização?

O `security-scan` reverte automaticamente todas as mudanças naquele ecossistema e continua com os demais. O ecossistema que falhou é reportado como "revertido" no relatório executivo.

### P: Posso usar o `security-scan` com um monorepo?

Sim. Use `scan.paths` para especificar quais subdiretórios varrer:

```
scan:
  auto_discover: false
  paths:
    - 'packages/frontend/'
    - 'packages/backend/'
```

### **P: O security-scan suporta yarn ou pnpm?**

Atualmente apenas npm ( package-lock.json ) e yarn v1 ( yarn.lock , apenas varredura de leitura) são suportados. pnpm ainda não é suportado.

### **P: Posso executar o security-scan sem Docker?**

Docker é obrigatório para executar os CLIs de ecossistema (npm, composer, pip) na fase de fix. O OSV Scanner também usa Docker por padrão, embora possa ser executado localmente com `runners.osv.runner: 'local'`. O modo `local` para runners de ecossistema está disponível mas não é recomendado e emite um aviso.

### **P: O que significa "autorização necessária" no relatório?**

Significa que o fix requer um bump de versão major (ex.: `v3` → `v4`) ou uma mudança na constraint declarada. Isso nunca é aplicado automaticamente. Para autorizar:

```
security-scan fix --authorize-breaking <ecossistema>
```

### **P: Como adiciono um novo ecossistema a uma configuração existente?**

Adicione uma nova entrada em `ecosystems` no `project-config.yml`:

```
ecosystems:
  - id: pip
    fixer: 'osv'
    validationCommands:
      - name: 'Tests'
        command: 'pytest'
```

### **P: Meus secrets estão seguros com o modo managed do SonarQube?**

No modo managed, a CLI gera um token temporário via API admin do SonarQube e o passa como argumento da CLI (não é gravado em disco). Os campos `sonar.login` / `sonar.password` no `sonar-project.properties` são removidos via uma cópia sanitizada temporária (sonar-scanner 5+ rejeita a mera presença desses campos).

### **P: Como faço para fixar a versão do OSV Scanner?**

```
scanners:
  osv:
    image: 'ghcr.io/google/osv-scanner:v1.9.0'
```

### **P: Posso gerar relatórios em inglês e português ao mesmo tempo?**

Não em uma única execução. Defina `report_language` como `en` ou `pt-br` . Para gerar ambos, execute `executive-report` duas vezes com arquivos de configuração diferentes.

**P: O security-scan é open source?**

Sim. Licenciado sob MIT.